

# 第9章 查找

Faculty of Information and Control Engineering

Huanliang Sun

Email: [sunhl@sjzu.edu.cn](mailto:sunhl@sjzu.edu.cn)

Office: Wu(戊2-306) 24690666

# 主要内容

- **第一讲**
  - 9.1 静态查找表**
  - 练习与作业**
- **第二讲 (第6章树中介绍)**
  - 9.2 动态查找表**
    - 9.2.1 二叉排序树和平衡二叉树**
    - 练习与作业**
- **第三讲**
  - 9.2.2 B\_树和B+树**
  - 练习与作业**
- **第四讲**
  - 9.3 哈希表**
  - 练习与作业**

# 查找表概述

- **查找**——也叫检索，是根据给定的某个值，在表中确定一个关键字等于给定值的记录或数据元素
- **查找表**：由具有同一类型（属性）的数据元素（记录）组成的集合。

| 学 号    | 姓 名 | 性 别 | 出生日期 |    |    | 来 源  | 总 分 | 录 取 专 业 |
|--------|-----|-----|------|----|----|------|-----|---------|
|        |     |     | 年    | 月  | 日  |      |     |         |
| 010983 | 赵剑平 | 男   | 1982 | 11 | 05 | 沈阳一中 | 593 | 计算机     |
| 010984 | 蒋伟峰 | 男   | 1982 | 09 | 12 | 锦州三中 | 601 | 计算机     |
| 010985 | 郭 娜 | 女   | 1983 | 01 | 25 | 盖县中学 | 598 | 计算机     |
|        |     |     |      |    |    |      |     |         |

高考成绩表示例

- **静态查找表**：仅对查找表进行查找，不能改变的表；
- **动态查找表**：对查找表除进行查找操作外，还要进行向表中插入数据元素，或删除表中数据元素的表。

—**数据项**（也称项或字段）：具有独立含义的标识单位，是数据不可分割的最小单位。

—**组合项**：由若干项、组合项构成，表中“出生日期”就是组合项，它由“年”、“月”、“日”三项组成。

—**数据元素**（记录）：由若干数据项、组合项构成的数据单位，是在某一问题中作为整体进行考虑和处理的基本单位。

—**关键字**：数据元素（记录）中某个数据项或组合项的值，用它可以标识一个数据元素（记录）。主关键字，次关键字

使用**主关键字**查找，查找结果唯一；若是用**次关键字**查找，需要查遍表中所有数据元素（记录），或在可以肯定查找失败时，才能结束查找过程。

# 9.1 静态查找表

## 9.1.1 顺序表的查找

- **查找过程**：对给定的一关键字 $K$ ，从线性表的一端开始，逐个进行记录的关键字和 $K$ 的比较，直到找到关键字等于 $K$ 的记录或到达表的另一端。
- **查找方向**：从前向后、从后向前
- **只能采取顺序查找情况**：线性表为无序表、有序线性表但采用的是链式存储结构。

# 顺序查找的算法

```
int Search_seq(SSTable ST, KeyType key)
```

```
{ int i=ST.length;
```

```
  ST[0].key=key;
```

```
  while(ST[i].key!=key) i- -; /*从表尾往前查*/
```

```
  return i;
```

```
}
```

使用了监视哨，查找过程中，不用每一步都去判断是否查找结束。  
找到：返回元素在线性表中的存储位置；  
未找到：返回0。

| 0 | 1  | 2  | 3  | 4  | 5  | 6  | 7  |
|---|----|----|----|----|----|----|----|
|   | 10 | 20 | 40 | 80 | 30 | 60 | 25 |

(a) 初态

| 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  |
|----|----|----|----|----|----|----|----|
| 80 | 10 | 20 | 40 | 80 | 30 | 60 | 25 |

(b) **K=80** (return i=4)

| 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  |
|----|----|----|----|----|----|----|----|
| 90 | 10 | 20 | 40 | 80 | 30 | 60 | 25 |

(c) **K=90** (return i=0)

比较次数:

查找第n个元素: 1

查找第n-1个元素: 2

.....

查找第1个元素: n

查找第i个元素:  $n+1-i$

查找失败:  $n+1$

# 性能分析:

**平均查找长度ASL:**顺序查找方法确定数据元素在表中的位置所进行的关键字比较次数的期望值。

$$ASL = \sum p_i c_i$$

$p_i$  为表中第*i*个数据元素的查找概率  $\sum p_i = 1$

$c_i$  为表中第*i*个数据元素的关键字与给定值key相等时，按算法定位时关键字的比较次数。

设表中每个元素的查找概率相等  $p_i = \frac{1}{n}$

$$\text{则 } ASL = \sum_{i=1}^n p_i c_i = \frac{1}{n} \sum_{i=1}^n i = \frac{1}{n} \cdot \frac{n(n+1)}{2} = \frac{n+1}{2}$$

不等概率查找时， $ASL_{ss}$  在  $P_n \geq P_{n-1} \geq \dots \geq P_2 \geq P_1$  时取极小值

当n较大时效率低，对表中数据元素的存储没有要求。

## 9.1.2 有序表的查找

- **有序表**即是表中数据元素按关键字升序或降序排列。
- **查找过程**：每次将待查记录所在区间缩小一半
- 顺序存储结构的有序表采用**折半查找（二分查找）**
  - 取**中间元素**作为比较对象，若给定值与中间元素的关键字相等，则查找成功；
  - 若给定值**小于中间元素**的关键字，则在中间元素的左半区继续查找；若给定值**大于中间元素**的关键字，则在中间元素的右半区继续查找。
  - **不断重复**上述查找过程，直到查找成功，或所查找的区域无数据元素，查找失败。

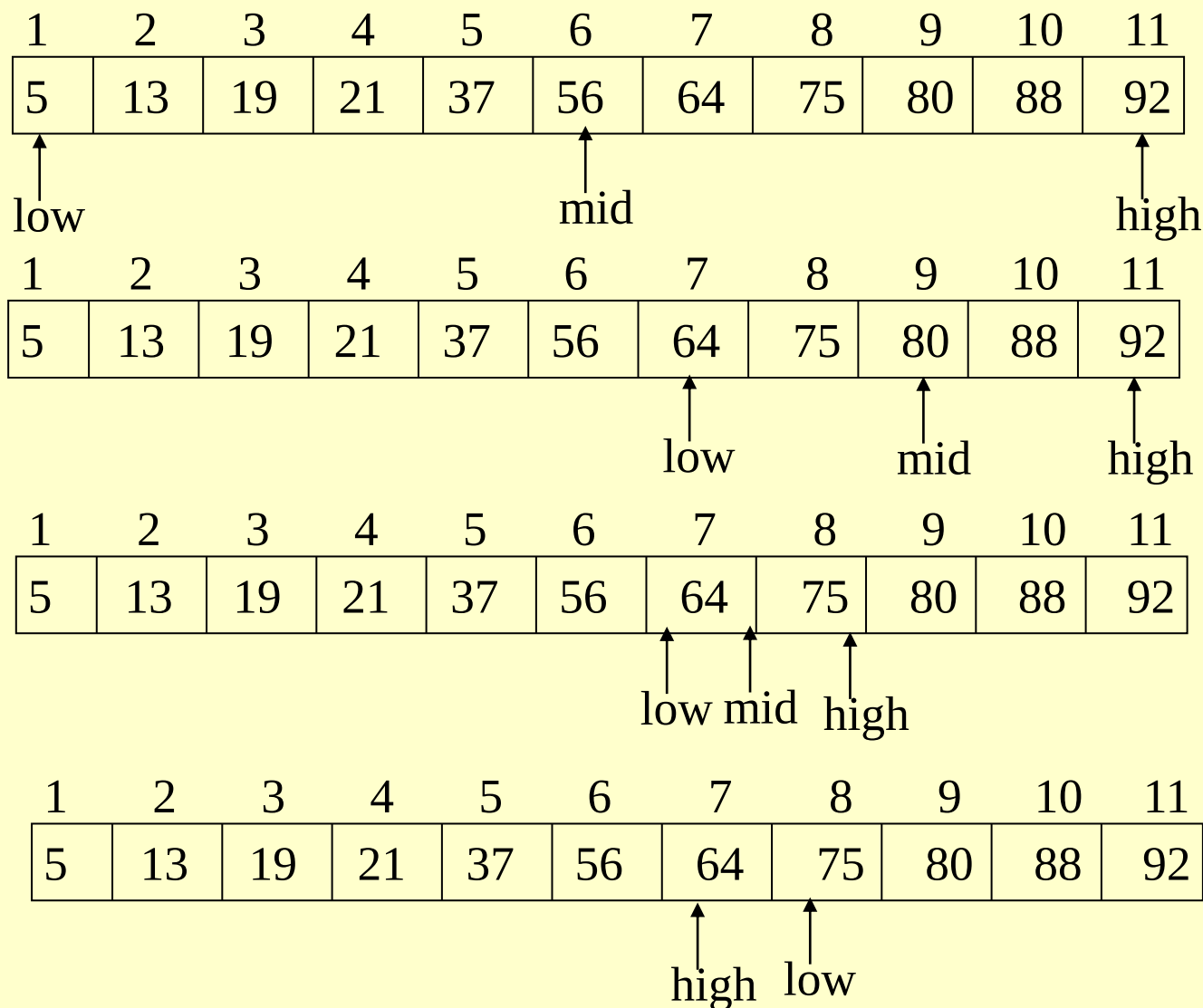


## – 算法实现

- 设表长为 $n$ ， $low$ 、 $high$ 和 $mid$ 分别指向待查元素所在区间的上界、下界和中点， $key$ 为给定值
- 初始时，令 $low=1, high=n, mid=\lfloor (low+high)/2 \rfloor$
- 让 $k$ 与 $mid$ 指向的记录比较
  - 若 $key == R[mid]$ ，查找成功
  - 若 $key < R[mid]$ ，则 $high=mid-1$
  - 若 $key > R[mid]$ ，则 $low=mid+1$
- 重复上述操作，直至 $low > high$ 时，查找失败



找70



◆搜索成功:  $key == R[mid]$

◆搜索不成功:  $low > high$

```
int binarySearchIteration(ElemType *R,int low,int
high,int key)
{
    int mid;
    while(low<=high) {
        mid=(low+high)/2;
        if(R[mid]>key)
            high=mid-1;
        else if(R[mid]<key)
            low=mid+1;
        else//find the key
            return mid;
    } //the array does not contain the key
    return -1;
}
```

```
int binarySearchRecursion(ElemType *R, int low, int high, int key)
{
    int mid;
    mid=low+(high-low)/2;
    if(low>high)
        return -1;
    else{
        if(R[mid]==key)
            return mid;
        else if(R[mid]>key)
            return binarySearchRecursion(R, low, mid - 1, key);
        else
            return binarySearchRecursion(R, mid + 1, high, key);
    }
}
```

## Experiment 9-1 折半查找（二分查找）

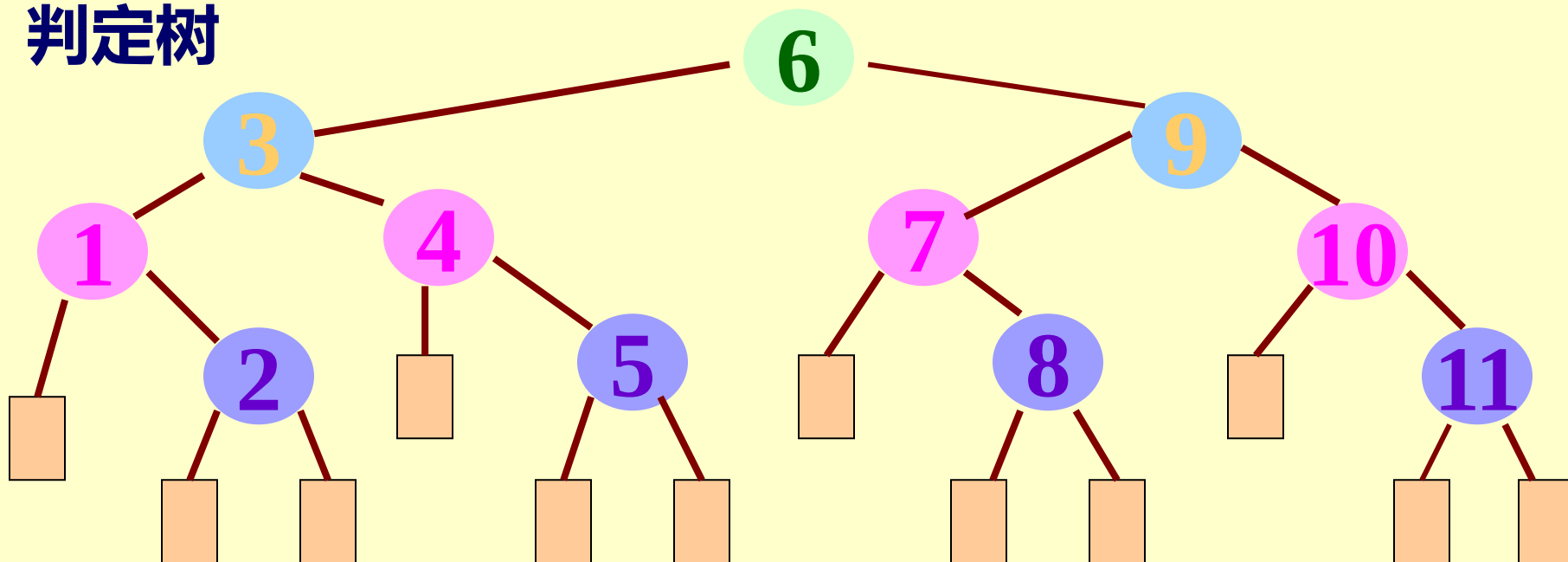
- 输入有序数，实现折半查找（二分查找）算法.

# 分析折半查找的平均查找长度

n=11

| i  | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
|----|---|---|---|---|---|---|---|---|---|----|----|
| Ci | 3 | 4 | 2 | 3 | 4 | 1 | 3 | 4 | 2 | 3  | 4  |

## 判定树



查找成功时平均查找长度？

查找失败时平均查找长度？

## □ 算法评价

- 判定树：描述查找过程的二叉树叫判定树
- 有 $n$ 个结点的判定树的深度为 $\lfloor \log_2 n \rfloor + 1$
- 折半查找在查找过程比较次数最多不超过其判定树的深度
- 判定是AVL树
- 折半查找的ASL

## 成功时

设表长 $n = 2^h - 1$ ,  $h = \log_2(n + 1)$ , 即判定树是深度为 $h$ 的满二叉树

设表中每个记录的查找概率相等 $p_i = \frac{1}{n}$

$$\text{则: } ASL = \sum_{i=1}^n p_i c_i = \frac{1}{n} \sum_{i=1}^n c_i = \frac{1}{n} \sum_{j=1}^h j \cdot 2^{j-1} = \frac{n+1}{n} \log_2(n+1) - 1 \approx \log_2(n+1) - 1$$

$j$ 为判定树的层,  $j=1$ 为1个结点,  $j=2$ 为2个结点, ...

折半查找的效率比顺序查找高, 前提要求**顺序表**有序



# 练习

设有序顺序表中的元素依次为

015, 090, 152, 174, 273, 501, 507, 510, 551, 610, 675, 763, 895, 906。

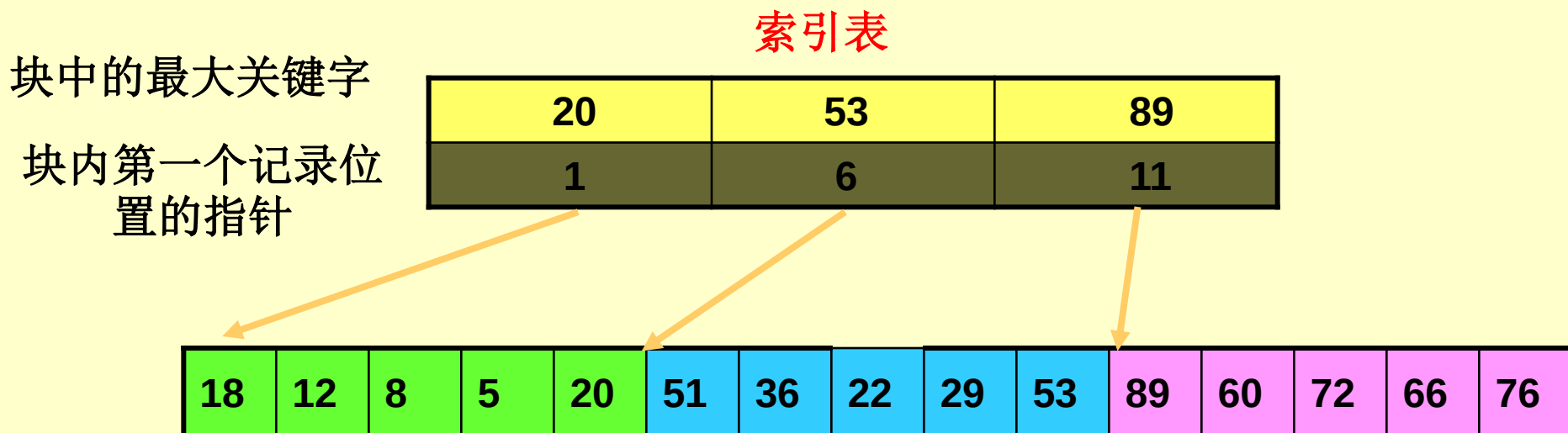
试画出对其进行折半查找的判定树；

若查找273或682的元素，将依次与表中的哪些元素比较？

计算查找成功的平均查找长度和查找失败的平均查找长度。

## 9.1.4 索引顺序表的查找（分块查找）

- **查找过程**：将表分成几块，块内无序，块间有序；先确定待查记录所在块，再在块内查找
- **适用条件**：分块有序表
- **算法实现**
  - 用**数组**存放待查记录,每个数据元素至少含有关键字域
  - 建立**索引表**，每个索引表结点含有最大关键字域和指向本块第一个结点的指针



## — 分块查找方法评价

$$ASL_{bs} = L_b + L_w$$

其中： $L_b$  —— 查找索引表确定所在块的平均查找长度

$L_w$  —— 在块中查找元素的平均查找长度

若将表长为 $n$ 的表平均分成 $b$ 块，每块含 $s$ 个记录，并设表中每个记录的查找概率相等，则：

(1)用顺序查找确定所在块：
$$ASL_{bs} = \frac{1}{b} \sum_{j=1}^b j + \frac{1}{s} \sum_{i=1}^s i = \frac{b+1}{2} + \frac{s+1}{2} = \frac{1}{2} \left( \frac{n}{s} + s \right) + 1$$

(2)用折半查找确定所在块：
$$ASL_{bs} \approx \log_2 \left( \frac{n}{s} + 1 \right) + \frac{s}{2}$$

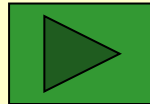
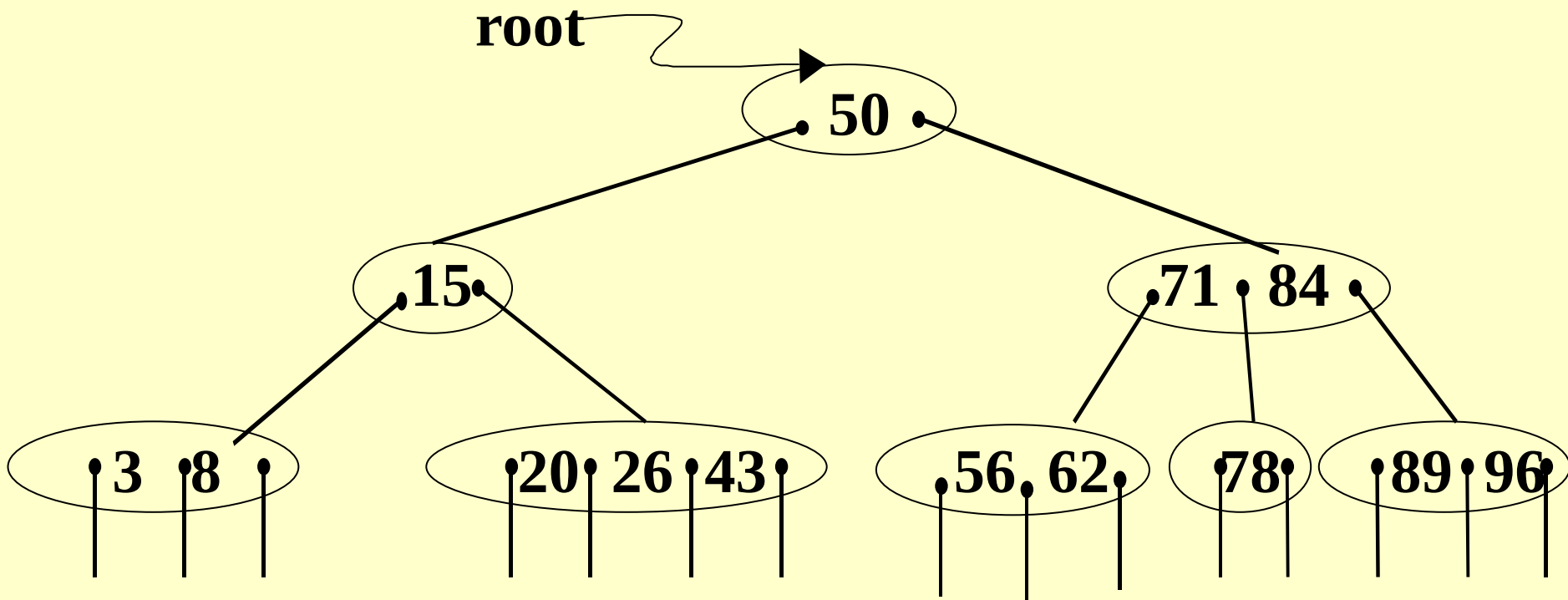
## 查找方法比较

|      | 顺序查找           | 折半查找   | 分块查找           |
|------|----------------|--------|----------------|
| ASL  | 最大             | 最小     | 两者之间           |
| 表结构  | 有序表、无序表        | 有序表    | 分块有序表          |
| 存储结构 | 顺序存储结构<br>线性链表 | 顺序存储结构 | 顺序存储结构<br>线性链表 |

## 9.2.2 B 树和 B<sup>+</sup>树

### 一、B树及其查找

B树是一种 平衡 的多路 查找 树:



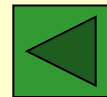
**定义：一棵m阶的B-树，或者为空树，或为满足下列特性的m叉树**

- (1) 树中每个结点至多有m棵子树；**
- (2) 若根结点不是叶子结点，则至少有两棵子树；**
- (3) 除根结点外所有非终端结点至少有 $\lceil m/2 \rceil$ 棵子树；**
- (4) 所有的非终端结点中包含以下信息数据：**

**$(n, A_0, K_1, A_1, K_2, \dots, K_n, A_n)$**

**其中：  $K_i$  ( $i=1,2,\dots,n$ ) 为关键字，且  $K_i < K_{i+1}$ ，**

**$A_i$ 为指向子树根结点的指针( $i=0,1,\dots,n$ )，且指针 $A_{i-1}$ 所指子树中所有结点的关键字均小于 $K_i$ ( $i=1\dots n$ )， $A_n$ 所指子树中所有结点的关键字均大于 $K_n$ ，  $n$ 为关键字的个数，范围是 $\lceil m/2 \rceil - 1 \leq n \leq m - 1$ 。**



(5) 所有的叶子结点都出现在同一层次上，并且不带信息（可以看作是外部结点或查找失败的结点，实际上这些结点不存在，指向这些结点的指针为空）。

## 2.查找过程：

从根结点出发，沿指针**搜索结点**和**在结点内进行顺序（或折半）查找**两个过程交叉进行。

若**查找成功**，则**返回指向被查关键字所在结点的指针和关键字在结点中的位置**；

若**查找不成功**，则**返回插入位置**。

# 结论:

$$\sum_{i=0}^N A^i = \frac{A^{N+1} - 1}{A - 1}$$

## ➤ m阶高度为h的B-树

最多结点数:  $(m^h - 1) / (m - 1)$

最多关键字:  $m^h - 1$

最少结点数:  $1 + 2 * (\lceil m/2 \rceil^{(h-1)} - 1) / (\lceil m/2 \rceil - 1)$

最少关键字:  $2(\lceil m/2 \rceil^{(h-1)} - 1)$ , 当m=3时,  $2^5 - 1 = 31$

## ➤ m阶N 个关键字的B-树

最大高度:  $H \leq \log_{\lceil m/2 \rceil}((N+1)/2) + 1$

最小高度:  $H \geq \log_m(N+1)$

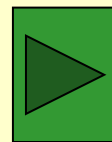
一次查找访问的结点个数不超过:  $\log_{\lceil m/2 \rceil}((N+1)/2) + 1$



### 3. 插入

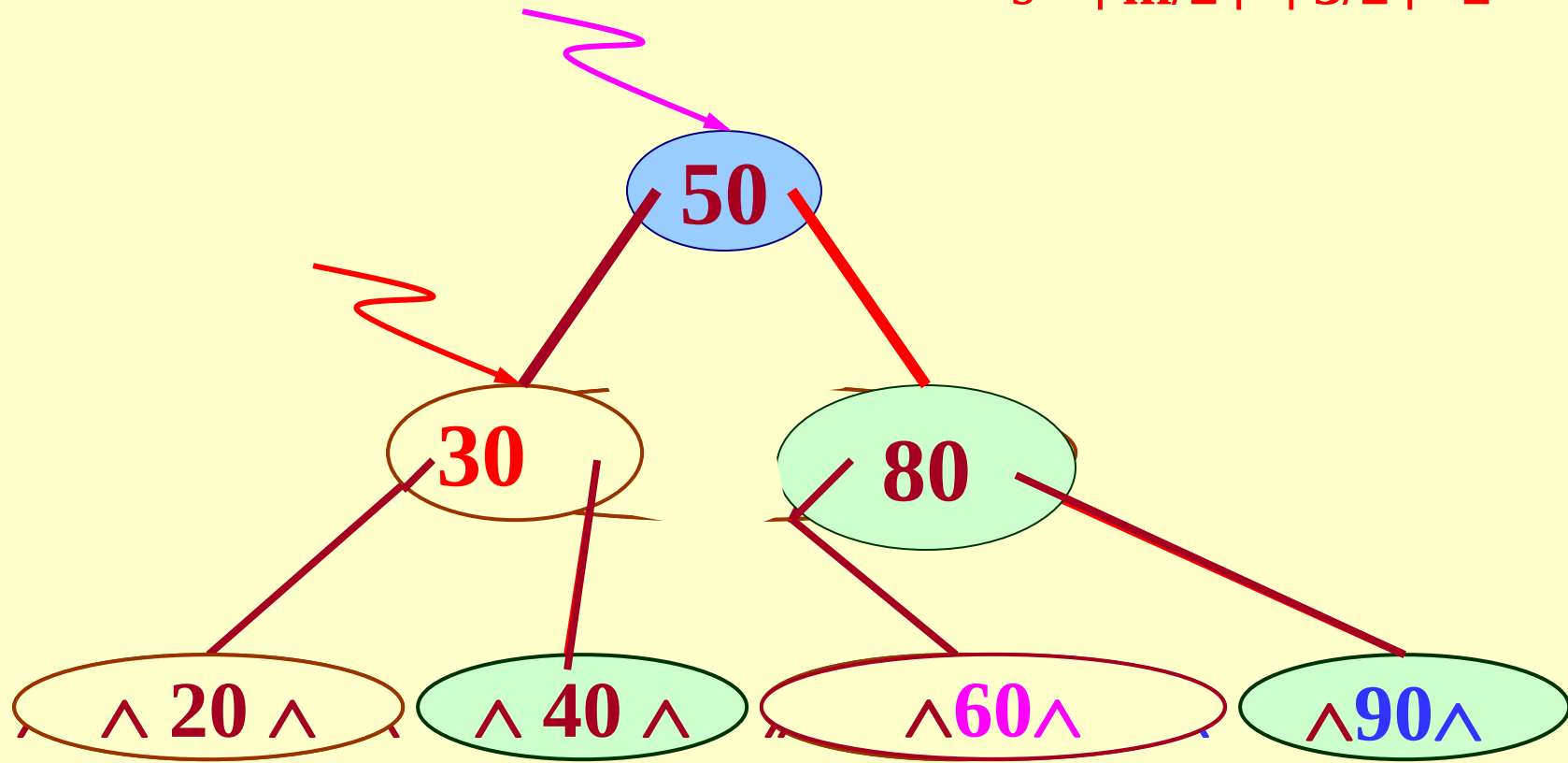
在查找不成功之后，需进行插入。显然，关键字插入的位置必定在最下层的非叶结点，有下列几种情况：

- 1) 插入后，该结点的关键字个数  $n < m$ ，不修改指针；
- 2) 插入后，该结点的关键字个数  $n = m$ ，则需进行“结点分裂”，令  $s = \lceil m/2 \rceil$ ，在原结点中保留  $(A_0, K_1, \dots, K_{s-1}, A_{s-1})$ ；  
建新结点  $(A_s, K_{s+1}, \dots, K_n, A_n)$ ；  
将  $(K_s, p)$  插入双亲结点；K 是关键字，A 为指针。
- 3) 若双亲为空，则建新的根结点。

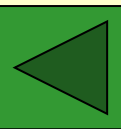


### 3 阶B-树的示意图

$$s = \lceil m/2 \rceil = \lceil 3/2 \rceil = 2$$



插入关键字 = 60, 90, 30,



## 4. 删除

和插入的考虑相反，首先必须找到待删关键字所在结点，并且要求删除之后，结点中关键字的个数不能小于 $\lceil m/2 \rceil - 1$ ，否则，要从其左(或右)兄弟结点“借调”关键字，若其左和右兄弟结点均无关键字可借(结点中只有最少量的关键字)，则必须进行结点的“合并”。

根据关键字删除记录，如果B树中记录中不存在对应关键字的记录，删除失败。

1) 如果当前需要删除的关键字位于非叶子结点上，则用后继关键字（指后继记录）覆盖要删除的关键字，然后在后继key所在的子支中删除该后继关键字。此时后继关键字一定位于叶子结点上，这个过程和二叉搜索树删除结点的方式类似。执行第2步

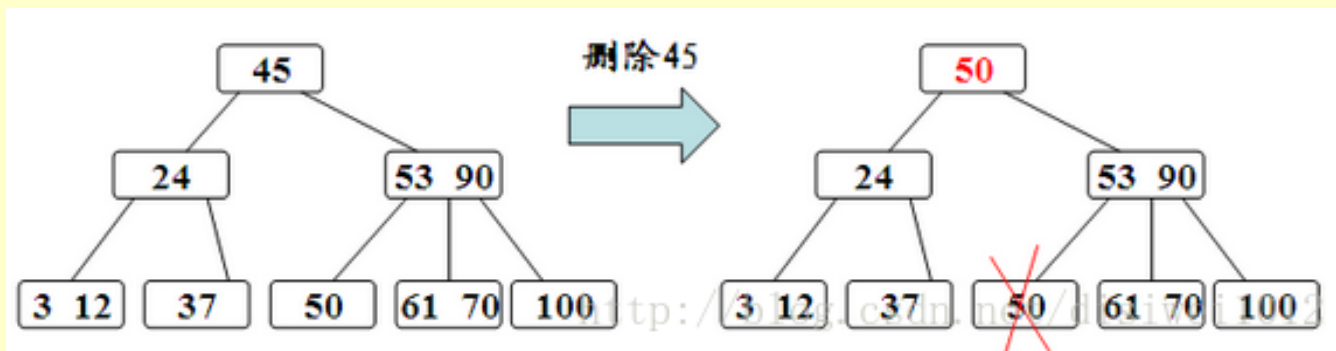
2) 该结点key个数大于等于 $\lceil m/2 \rceil - 1$ ，直接删除。否则执行第3步。

3) 如果兄弟够借，即兄弟结点key个数大于 $\lceil m/2 \rceil - 1$ ，则父结点中的key下移到该结点，兄弟结点中的一个key上移，删除操作结束。

否则，即兄弟不够借，将父结点中的key下移与当前结点及它的兄弟结点中的key合并，形成一个新结点。原父结点中的key的两个孩子指针就变成了一个孩子指针，指向这个新结点。然后当前结点的指针指向父结点，重复上第2步。

有些结点可能即有左兄弟，又有右兄弟，那么任意选择一个兄弟结点进行操作。

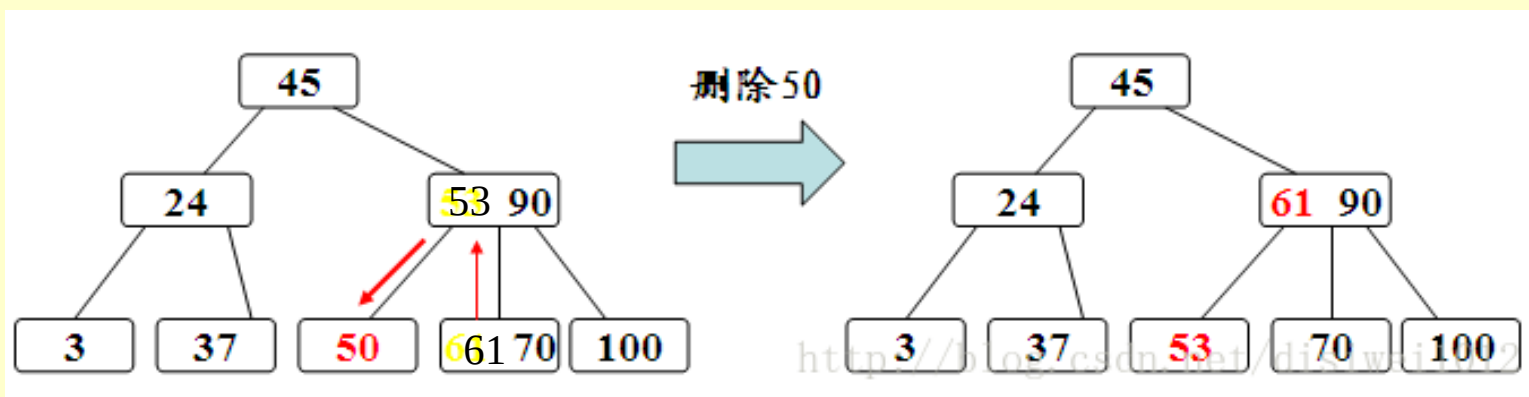
(1) 如果当前需要删除的关键字位于非叶子结点上，则用后继关键字覆盖要删除的关键字，然后在后继key所在的子支中删除该后继关键字。然后考虑删除叶子结点的情况。



$m=3$

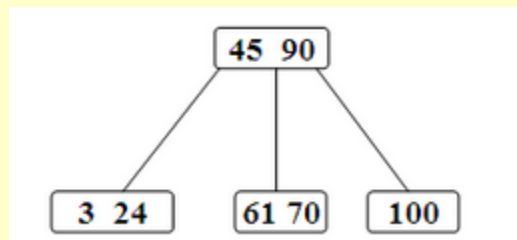
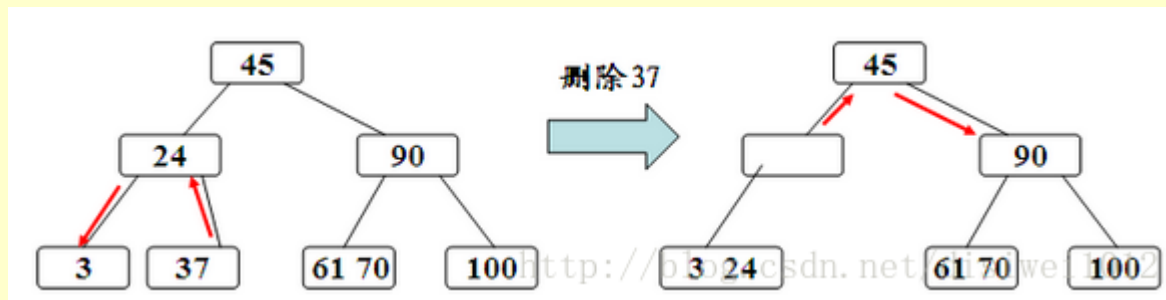
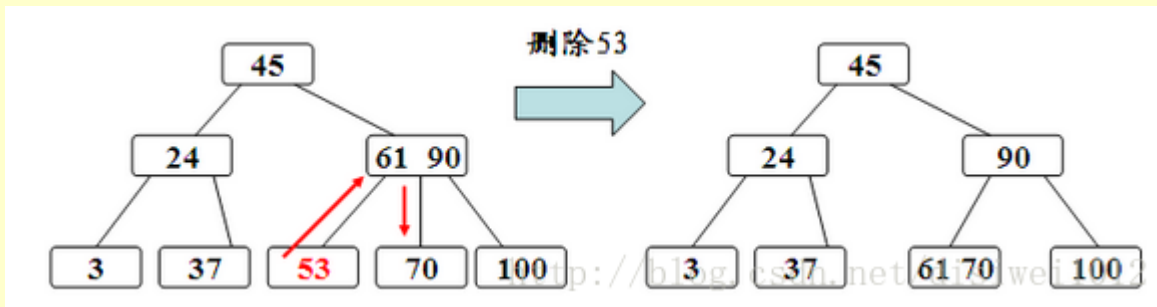
(2) 结点key个数大于等于 $\lceil m/2 \rceil - 1$ ，直接删除。如删除12

(3) 兄弟够借，被删关键字 $K_i$ 所在结点的关键字数目等于 $\lceil m/2 \rceil - 1$



(3) 兄弟不够借，被删关键字 $K_i$ 所在结点和其相邻兄弟结点中的的关键字数目均等于 $\lceil m/2 \rceil - 1$ ，假设该结点有右兄弟，且其右兄弟结点地址由其双亲结点指针 $A_i$ 所指。则在删除关键字之后，它所在结点的剩余关键字和指针，加上双亲结点中的关键字 $K_i$ 一起，合并到 $A_i$ 所指兄弟结点中（若无右兄弟，则合并到左兄弟结点中）。如果因此使双亲结点中的关键字数目少于 $\lceil m/2 \rceil - 1$ ，则依次类推。

$m=3$

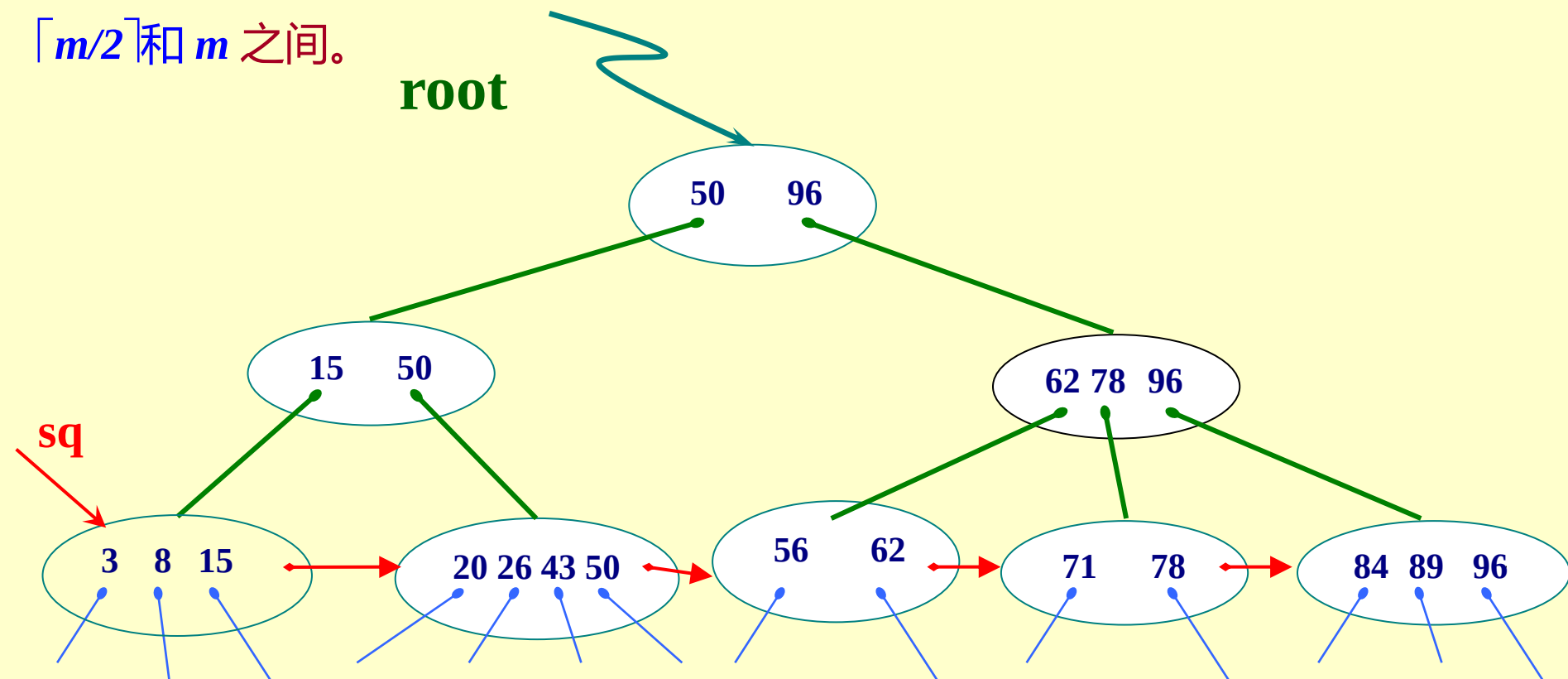


## 二、B+树

※有 $n$ 个子树的叶子结点中含有  $n$  个关键字和  $n$  个指向记录的指针；并且，所有**叶子**结点彼此相**链接**依照**关键字**构成一个有序链表，其头指针指向含最小关键字的结点；

※ 每个非叶结点中的关键字  $K_i$  即为其相应指针  $A_i$  所指子树中关键字的最大值；

※ 所有叶子结点都处在同一层次上，每个叶子结点中关键字的个数均介于  $\lceil m/2 \rceil$  和  $m$  之间。



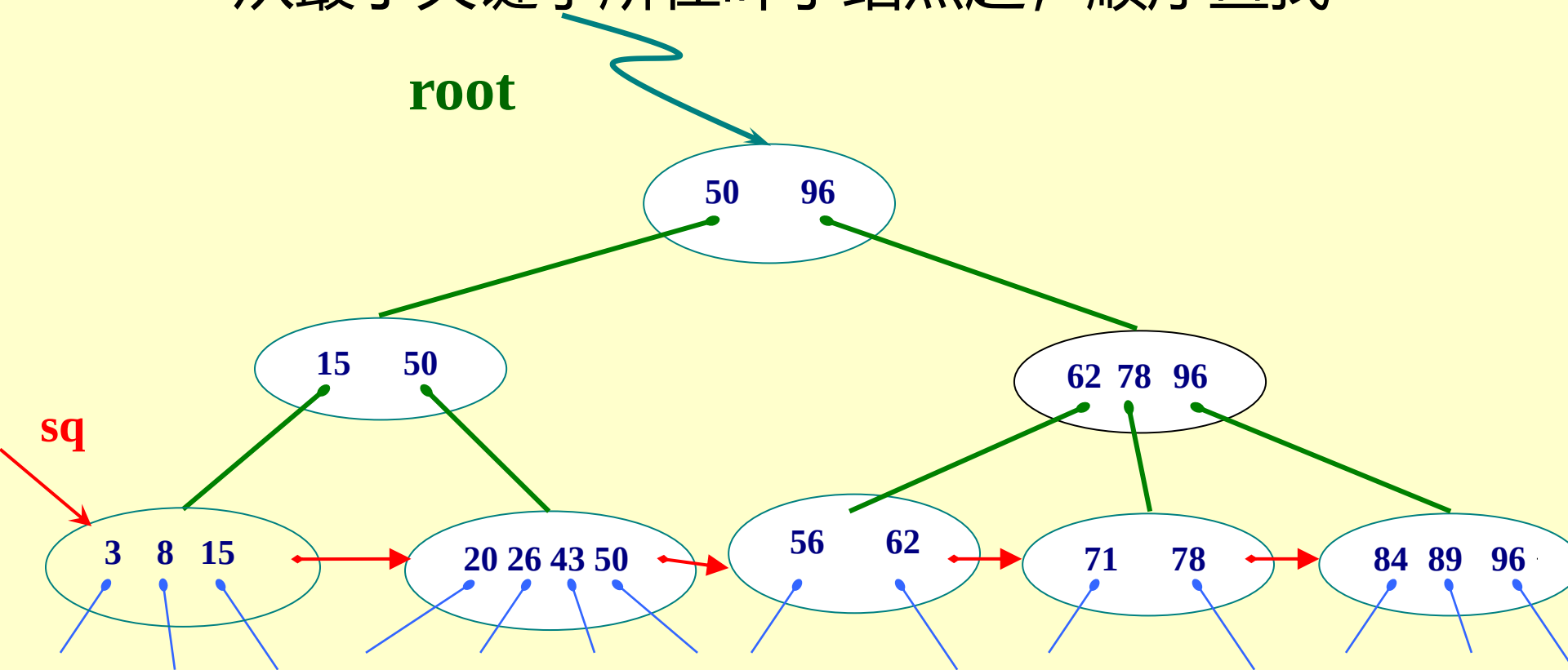
## B<sup>+</sup>树的结构特点:

- 有 $n$ 个子树的叶子结点中含有  $n$  个关键字和  $n$  个指向记录的指针;
- 所有**叶子**结点相**链接依照关键字**构成一个有序链表, 其头指针指向含最小关键字的结点;
- 每个非叶结点中的关键字  $K_i$  即为其相应指针  $A_i$  所指子树中关键字的最大值;
- 所有叶子结点都处在**同一层次**上, 每个叶子结点中关键字的个数均介于  $\lceil m/2 \rceil$  和  $m$  之间。



## B+树的查找：

- 从根结点开始沿子树一直查找到叶子结点的关键字由该关键字指示的指针查找记录
- 从最小关键字所在叶子结点起，顺序查找





# 练习

给定一组关键字{18,32,48,54,62,70,80}，给出创建一棵3阶B树的过程。

```

graph TD
    Root["12 25"]
    Leaf1["05 10"]
    Leaf2["15 19 22"]
    Leaf3["26 28"]
    Root --- Leaf1
    Root --- Leaf2
    Root --- Leaf3
  
```

05 10      15 19 22      26 28

例 1 押新D同 并且右

到一棵新B树，其最右

45

$$\begin{array}{|c|} \hline 17 \ 35 \\ \hline \end{array} \quad \begin{array}{|c|} \hline 55 \ 65 \\ \hline \end{array}$$

10      21      37      47      60 62      7

少是 ( )

不足( )

一、当日在日 ( )

这个数最多是（ ）。